Programming Fundamentals

Dafny - week 9

A. Meijster

University of Groningen

university of groningen

Recall: While loops

With the Hoare-triple $\{P\}$ **while** B **do** S **end** $\{Q\}$ we denote a **while**-program fragment that establishes a state in which *postcondition* Q holds if the program was started in a state in which precondition P holds.

From

- $\text{vf} \in \mathbb{Z}$
- $J \wedge B \Rightarrow \text{vf} \geq 0$
- $\{J \wedge B \wedge \text{vf} = V\}\ S\ \{J \wedge \text{vf} < V\}$

we may conclude

$$\{J\}\ \textbf{while}\ B\ \textbf{do}\ S\ \textbf{end}\ \{J \wedge \neg B\}$$

university of groningen

Example: From recurrence to a while-loop

The function f is defined by the recurrence:

$$\begin{array}{lcl} y \leq 0 & \Rightarrow & f(y,z) = z \\ y > 0 & \Rightarrow & f(y,z) = 10 \cdot f(y \textbf{ div } 10, z) + y \textbf{ mod } 10 \end{array}$$

Find a command S that satisfies the specification:

$$\begin{array}{l} \textbf{var } y,\ z :\ \mathbb{Z}; \\ \quad \{P :\ Z = f(y,z)\} \\ S \\ \quad \{Q :\ Z = z\} \end{array}$$

We introduce auxiliary variables m and n, and $J :\ Z = m \cdot f(y,z) + n$
Initialization is easy:

$$\begin{array}{l} \quad \{P : Z = f(y,z)\} \\ \quad\quad (* \text{ calculus } *) \\ \quad \{Z = 1 \cdot f(y,z) + 0\} \\ m := 1;\ n := 0; \\ \quad \{J :\ Z = m \cdot f(y,z) + n\} \end{array}$$

university of groningen

Example: From recurrence to a while-loop (Continued)

$$\begin{array}{lcl} y \leq 0 & \Rightarrow & f(y,z) = z \\ y > 0 & \Rightarrow & f(y,z) = 10 \cdot f(y \textbf{ div } 10, z) + y \textbf{ mod } 10 \\ J :\ Z & = & m \cdot f(y,z) + n \\ Q :\ Z & = & z \end{array}$$

The next step is to choose the guard B.

From the definition of f we see that we can determine $f(y,z)$ directly if $y \leq 0$.
Therefore, we choose the guard $B : \neg(y \leq 0) \quad \equiv \quad y > 0$.

We need (active) finalization to establish Q:

$$\begin{array}{l} \quad \{J \ \wedge \ \neg B\} \\ \quad \{Z = m \cdot f(y,z) + n \ \wedge \ y \leq 0\} \\ \quad\quad (* \text{ definition } f *) \\ \quad \{Z = m \cdot z + n\} \\ z := m * z + n; \\ \quad \{Z = z\} \end{array}$$

university of groningen

Example: From recurrence to a while-loop (Continued)

variant function: The guard is $y > 0$, so we need to decrease y until $y \leq 0$.
We choose vf $= y \in \mathbb{Z}$.
Since $B \equiv \text{vf} > 0$, it is clear that $J \wedge B \Rightarrow \text{vf} \geq 0$.

For the body of the loop we find:

$\{J \wedge B \wedge \text{vf} = V\}$
$\{Z = m \cdot f(y,z) + n \wedge y > 0 \wedge y = V\}$
$(* \; y = V > 0 \Rightarrow y \; \textbf{div} \; 10 < V$; definition $f \; *)$
$\{Z = m \cdot (10 \cdot f(y \; \textbf{div} \; 10, z) + y \; \textbf{mod} \; 10) + n \wedge y \; \textbf{div} \; 10 < V\}$
$(*$ calculus $*)$
$\{Z = 10 \cdot m \cdot f(y \; \textbf{div} \; 10, z) + m \cdot (y \; \textbf{mod} \; 10) + n \wedge y \; \textbf{div} \; 10 < V\}$
$n := m * (y \; \textbf{mod} \; 10) + n;$
$\{Z = 10 \cdot m \cdot f(y \; \textbf{div} \; 10, z) + n \wedge y \; \textbf{div} \; 10 < V\}$
$m := 10 * m;$
$\{Z = m \cdot f(y \; \textbf{div} \; 10, z) + n \wedge y \; \textbf{div} \; 10 < V\}$
$y := y \; \textbf{div} \; 10;$
$\{Z = m \cdot f(y,z) + n \wedge y < V\}$
$\{J \wedge \text{vf} < V\}$

university of groningen

Example: From recurrence to a while-loop (Continued)

Conclusion: we derived the following program fragment:

var $n, m, y, z : \mathbb{Z};$
$\{P : Z = f(y,z)\}$
$m := 1;$
$n := 0;$
$\{J : Z = m \cdot f(y,z) + n\}$
$(* \; \text{vf} = y \; *)$
while $y > 0$ **do**
$n := m * (y \; \textbf{mod} \; 10) + n;$
$m := 10 * m;$
$y := y \; \textbf{div} \; 10;$
end;
$z := m * z + n;$
$\{Q : z = Z\}$

university of groningen

Example: From recurrence to a while-loop (Dafny)

Let us try to convert the program fragment into a Dafny program.
A first attempt might look like this:

```
ghost function f(y: int, z: int): int
{
  if y <= 0 then z else 10*f(y/10, z) + y%10
}

method ComputeF(y: int, z: int) returns (z: int)
ensures z == f(y,z)
{
  var m,n := 1,0;
  while y > 0
  {
    n := m*(y%10) + n;
    m := 10*m;
    y := y/10;
  }
  z := m*z + n;
}
```

Dafny reports

```
Error: Duplicate parameter name: z
```


Example: From recurrence to a while-loop (Dafny)

Ok, so we introduce a variable `r` to return the result.

```
method ComputeF(y: int, z: int) returns (r: int)
ensures r == f(y,z)
{
  var m,n := 1,0;
  while y > 0
  {
    n := m*(y%10) + n;
    m := 10*m;
    y := y/10;
  }
  r := m*z + n;
}
```

Dafny reports

```
Error: LHS of assignment must denote a mutable variable
```

The problem is that `y` is an in-parameter, and cannot be modified in the body of the method.

university of groningen

Example: From recurrence to a while-loop (Dafny)

So, we end up with:

```
ghost function f(y: int, z: int): int
{
  if y <= 0 then z else 10*f(y/10, z) + y%10
}
method ComputeF(y: int, z: int) returns (r: int)
ensures r == f(y,z)
{
  var m,n,a := 1,0,y;
  while a > 0
  invariant f(y,z) == m*f(a,z) + n
  {
    n := m*(a%10) + n;
    m := 10*m;
    a := a/10;
  }
  r := m*z + n;
}
```

Dafny reports

```
Dafny program verifier finished with 2 verified, 0 errors
```


Another example: recurrence to iteration

The function f is defined by the recurrence:

$$\begin{aligned} f(0) &= 0 \\ n > 0 \Rightarrow f(n) &= 5 \cdot f(n \textbf{ div } 3) + n \textbf{ mod } 4 \end{aligned}$$

Find a command S that satisfies the following specification:

$$\begin{array}{l} \textbf{var } n,\ x : \mathbb{Z}; \\ \quad \{P :\ n \geq 0\ \wedge\ Z = f(n)\} \\ S \\ \quad \{Q :\ Z = x\} \end{array}$$

Invariant and guard: We introduce a variable y.

$$J :\ Z = y \cdot f(n) + x\ \wedge\ n \geq 0$$

$$B :\ n \neq 0$$

$$\begin{array}{ll} & J\ \wedge\ \neg B \\ \equiv & \\ & Z = y \cdot f(n) + x\ \wedge\ n = 0 \\ \Rightarrow & \{\ f(0) = 0;\ \text{logic}\ \} \\ & Z = y \cdot 0 + x \\ \equiv & \{\ \text{calculus}\ \} \\ & Q : Z = x \end{array}$$

university of groningen

Recurrence to iteration (Continued)

$$J: \ Z = y \cdot f(n) + x \ \wedge \ n \geq 0$$

Initialization of the invariant is easy:

$$\begin{array}{l} \{P: Z = f(n) \ \wedge \ n \geq 0\} \\ \quad (* \text{ calculus } *) \\ \quad \{Z = 1 \cdot f(n) + 0 \ \wedge \ n \geq 0\} \\ x, y := 0, 1; \\ \quad \{J: Z = y \cdot f(n) + x \ \wedge \ n \geq 0\} \end{array}$$

Next, we need to choose a vf $\in \mathbb{Z}$. The invariant ensures that $n \geq 0$, and we chose $B: n \neq 0$. We will decrease n until $n = 0$. So, we choose vf $= n \in \mathbb{Z}$. Clearly, $J \ \wedge \ B \Rightarrow$ vf ≥ 0, since $vf \geq 0$ is a conjunct of J.

university of groningen

Recurrence to iteration (Continued)

$$\begin{array}{rcl} f(0) & = & 0 \\ n > 0 \Rightarrow f(n) & = & 5 \cdot f(n \ \textbf{div} \ 3) + n \ \textbf{mod} \ 4 \end{array}$$

For the body of the loop we find:

$$\begin{array}{l} \{J \ \wedge \ B \ \wedge \ \text{vf} = V\} \\ \{Z = y \cdot f(n) + x \ \wedge \ n \geq 0 \ \wedge \ n \neq 0 \ \wedge \ n = V\} \\ \quad (* \ n > 0 \Rightarrow f(n) = 5 \cdot f(n \ \textbf{div} \ 3) + n \ \textbf{mod} \ 4 \ \wedge \ 0 \leq n \ \textbf{div} \ 3 < n \ *) \\ \{Z = y \cdot (5 \cdot f(n \ \textbf{div} \ 3) + n \ \textbf{mod} \ 4) + x \ \wedge \ 0 \leq n \ \textbf{div} \ 3 < V\} \\ \quad (* \text{ calculus } *) \\ \{Z = 5 \cdot y \cdot f(n \ \textbf{div} \ 3) + y \cdot (n \ \textbf{mod} \ 4) + x \ \wedge \ 0 \leq n \ \textbf{div} \ 3 < V\} \\ x := y * (n \ \textbf{mod} \ 4) + x; \\ \quad \{Z = 5 \cdot y \cdot f(n \ \textbf{div} \ 3) + x \ \wedge \ 0 \leq n \ \textbf{div} \ 3 < V\} \\ y := 5 * y; \\ \quad \{Z = y \cdot f(n \ \textbf{div} \ 3) + x \ \wedge \ 0 \leq n \ \textbf{div} \ 3 < V\} \\ n := n \ \textbf{div} \ 3; \\ \quad \{Z = y \cdot f(n) + x \ \wedge \ 0 \leq n < V\} \\ \quad \{J \ \wedge \ \text{vf} < V\} \end{array}$$

university of groningen

Recurrence to iteration (Continued)

In conclusion, we derived the following program fragment:

$$
\begin{array}{l}
\textbf{var } x,\ y,\ n:\ \mathbb{Z};\\
\quad \{P:\ Z = f(n)\ \wedge\ n \geq 0\}\\
x, y := 0, 1;\\
\quad \{J:\ Z = y \cdot f(n) + x\ \wedge\ n \geq 0\}\\
\qquad (* \text{ vf} = n *)\\
\textbf{while } n \neq 0 \textbf{ do}\\
\quad x := y * (n \textbf{ mod } 4) + x;\\
\quad y := 5 * y;\\
\quad n := n \textbf{ div } 3;\\
\textbf{end};\\
\quad \{Q:\ x = Z\}
\end{array}
$$

university of groningen

Recurrence to iteration (Dafny version)

```
function f(n: nat): nat {
  if n == 0 then 0 else 5*f(n/3) + n%4
}

method computeF(m: nat) returns (x: nat)
ensures  x == f(m)
{
  var n, y;
  n,x,y := m,0,1;
  while n != 0
  invariant 0<=n && f(m)==y*f(n) + x
  {
    x := x + y*(n%4);
    y := 5*y;
    n := n/3;
  }
}
```

university of groningen

Exercise: Mid exam Program Correctness 2015

The function $f : \mathbb{N} \to \mathbb{N}$ is defined recursively as follows:

$$\begin{array}{rcl} n = 0 & \Rightarrow & f(n) = 1 \\ n > 0 \ \wedge \ n \ \textbf{mod} \ 2 = 0 & \Rightarrow & f(n) = 1 + 2 \cdot f(n \ \textbf{div} \ 2) \\ n > 0 \ \wedge \ n \ \textbf{mod} \ 2 = 1 & \Rightarrow & f(n) = 2 \cdot f(n \ \textbf{div} \ 2) \end{array}$$

Design a command S that satisfies $\{ \ P : \ n \geq 0 \ \wedge \ X = f(n) \ \} \ S \ \{ \ Q : \ X = a \ \}$

1. Invariant and guard: we choose the invariant $J : \quad X = a + b \cdot f(n) \ \wedge \ n \geq 0$.
   The recursion stops if $n = 0$, so we choose the guard $n \neq 0$.
   We will need active finalization to establish Q.

$$\begin{array}{l} \{ \ J \ \wedge \ \neg B \ \} \\ \{ \ n = 0 \ \wedge \ X = a + b \cdot f(n) \ \} \\ \{ \ X = a + b \ \} \\ a := a + b; \\ \{ \ Q : \ X = a \ \} \end{array}$$

2. Initialization:

$$\begin{array}{l} \{ \ P : \ n \geq 0 \ \wedge \ X = f(n) \ \} \\ \{ \ n \geq 0 \ \wedge \ X = 0 + 1 \cdot f(n) \ \} \\ a := 0; \ b := 1; \\ \{ \ J : \ n \geq 0 \ \wedge \ X = a + b \cdot f(n) \ \} \end{array}$$

3. Variant function: Choose vf $= n \in \mathbb{Z}$. Clearly, we have $J \ \wedge \ B \Rightarrow$ vf ≥ 0.

university of groningen

Exercise: Mid exam Program Correctness 2015 (Continued)

4. Body of the loop:

$$\begin{array}{l} \{ \ X = a + b \cdot f(n) \ \wedge \ n = V > 0 \ \} \\ \textbf{if} \ n \ \textbf{mod} \ 2 = 0 \ \textbf{then} \\ \quad \{ \ n \ \textbf{mod} \ 2 = 0 \ \wedge \ X = a + b \cdot f(n) \ \wedge \ n = V > 0 \ \} \\ \quad \quad (* \ n > 0 \ \wedge \ n \ \textbf{mod} \ 2 = 0 \Rightarrow f(n) = 1 + 2 \cdot f(n \ \textbf{div} \ 2) \ *) \\ \quad \{ \ X = a + b \cdot (1 + 2 \cdot f(n \ \textbf{div} \ 2)) \ \wedge \ n = V > 0 \ \} \\ \quad \{ \ X = a + b + 2 \cdot b \cdot f(n \ \textbf{div} \ 2) \ \wedge \ n = V > 0 \ \} \\ \quad a := a + b; \\ \quad \{ \ X = a + 2 \cdot b \cdot f(n \ \textbf{div} \ 2) \ \wedge \ n = V > 0 \ \} \\ \textbf{else} \\ \quad \{ \ n \ \textbf{mod} \ 2 = 1 \ \wedge \ X = a + b \cdot f(n) \ \wedge \ n = V > 0 \ \} \\ \quad \quad (* \ n > 0 \ \wedge \ n \ \textbf{mod} \ 2 = 1 \Rightarrow f(n) = 2 \cdot f(n \ \textbf{div} \ 2) \ *) \\ \quad \{ \ X = a + 2 \cdot b \cdot f(n \ \textbf{div} \ 2) \ \wedge \ n = V > 0 \ \} \\ \quad \textbf{skip;} \\ \quad \{ \ X = a + 2 \cdot b \cdot f(n \ \textbf{div} \ 2) \ \wedge \ n = V > 0 \ \} \\ \textbf{end}; \quad (* \ \text{collect branches} \ *) \\ \quad \{ \ X = a + 2 \cdot b \cdot f(n \ \textbf{div} \ 2) \ \wedge \ n = V > 0 \ \} \\ b := 2 * b; \\ \quad \{ \ X = a + b \cdot f(n \ \textbf{div} \ 2) \ \wedge \ n = V > 0 \ \} \\ \quad \quad (* \ \text{calculus}; \ n > 0 \Rightarrow 0 \leq n \ \textbf{div} \ 2 < n \ *) \\ \quad \{ \ X = a + b \cdot f(n \ \textbf{div} \ 2) \ \wedge \ n \ \textbf{div} \ 2 \geq 0 \ \wedge \ n \ \textbf{div} \ 2 < V \ \} \\ n := n \ \textbf{div} \ 2; \\ \quad \{ \ X = a + b \cdot f(n) \ \wedge \ n \geq 0 \ \wedge \ n < V \ \} \end{array}$$

university of groningen

Exercise: Mid exam Program Correctness 2015 (Continued)

$$
\begin{array}{l}
\textbf{var } a,\ b,\ n\ :\ \mathbb{N};\\
\quad \{\ P:\ n \geq 0\ \wedge\ X = f(n)\ \}\\
a := 0;\ b := 1;\\
\quad \{\ J:\ X = a + b \cdot f(n)\ \wedge\ n \geq 0\ \}\\
\qquad (*\ \mathrm{vf}:\ n\ *)\\
\textbf{while } n \neq 0 \textbf{ do}\\
\quad \textbf{if } n \textbf{ mod } 2 = 0 \textbf{ then}\\
\qquad a := a + b;\\
\quad \textbf{end};\\
\quad b := 2 * b;\\
\quad n := n \textbf{ div } 2;\\
\textbf{end};\\
a := a + b;\\
\quad \{\ Q:\ X = a\ \}
\end{array}
$$

university of groningen

Example: Mid exam Program Correctness 2015 (in Dafny)

```
include "io.dfy"

function f(n: nat): nat {
  if n == 0 then 1
  else if n%2 == 0 then 1 + 2*f(n/2)
       else 2*f(n/2)
}

method F(m: nat) returns (a: nat)
ensures  a == f(m)
{
  var n, b: nat;
  a,b,n := 0,1,m;
  while n != 0
  invariant 0<=n && f(m)==a + b*f(n)
  {
    if n%2==0 {
      a := a + b;
    }
    b := 2*b;
    n := n/2;
  }
  a := a + b;
}

method Main() {
  var n := IO.ReadNat();
  var f := F(n);
  print "f(", n, ")=", f, ".\n";
}
```

university of groningen

Dijkstra's Fusc function: from recurrence to a while loop

$$\begin{array}{rcl} f(0) = 0 & \wedge & f(1) = 1 \\ f(2 \cdot n) & = & f(n) \\ f(2 \cdot n + 1) & = & f(n) + f(n+1) \end{array}$$

We aim for a program fragment that satisfies the following specification:

var $n,\ x : \mathbb{Z}$;
$\{P : n \geq 0 \ \wedge \ Z = f(n)\}$
S;
$\{Q : x = Z\}$

1. Invariant and guard:

$$\begin{array}{ll} J: & n \geq 0 \ \wedge \ Z = y \cdot f(n) + x \cdot f(n+1) \\ B: & n \neq 0 \end{array}$$

university of groningen

Dijkstra's Fusc function (continued)

$$\begin{array}{rcl} f(0) = 0 & \wedge & f(1) = 1 \\ f(2 \cdot n) & = & f(n) \\ f(2 \cdot n + 1) & = & f(n) + f(n+1) \end{array}$$

2. Initialization: Find a command T_0 such that
$\{n \geq 0 \ \wedge \ Z = f(n)\}\ T_0\ \{n \geq 0 \ \wedge \ Z = y \cdot f(n) + x \cdot f(n+1)\}$

$\{P : n \geq 0 \ \wedge \ Z = f(n)\}$
$(*$ calculus $*)$
$\{n \geq 0 \ \wedge \ Z = 1 \cdot f(n) + 0 \cdot f(n+1)\ \}$
$y := 1;\ x := 0;$
$\{J : n \geq 0 \ \wedge \ Z = y \cdot f(n) + x \cdot f(n+1)\}$

3. Variant function: vf $= n \in \mathbb{Z}$.
The invariant contains the conjunct $n \geq 0$, so $J \ \wedge \ B \Rightarrow$ vf ≥ 0.

university of groningen

Fusc: [4] loop body $\{J \wedge B \wedge \mathrm{vf} = V\}\ S\ \{J \wedge \mathrm{vf} < V\}$

$\{0 < n = V \wedge Z = y \cdot f(n) + x \cdot f(n+1)\}$
$\{0 < n = V \wedge Z = y \cdot f(2 \cdot (n \textbf{ div } 2) + n \textbf{ mod } 2) + x \cdot f(2 \cdot (n \textbf{ div } 2) + n \textbf{ mod } 2 + 1)\}$
var $k := n$ **div** 2;
$\{0 \le k < V \wedge Z = y \cdot f(2 \cdot k + n \textbf{ mod } 2) + x \cdot f(2 \cdot k + n \textbf{ mod } 2 + 1)\}$
if n **mod** $2 = 0$ **then**
  $\{0 \le k < V \wedge Z = y \cdot f(2 \cdot k) + x \cdot f(2 \cdot k + 1)\}$
  $(*$ logic; definition $f(n)$ $*)$
  $\{0 \le k < V \wedge Z = y \cdot f(k) + x \cdot (f(k) + f(k+1))\}$
  $\{0 < n = V \wedge Z = (x + y) \cdot f(k) + x \cdot f(k+1)\}$
 $y := x + y$;
  $\{0 \le k < V \wedge Z = y \cdot f(k) + x \cdot f(k+1)\}$
else
  $\{0 \le k < V \wedge Z = y \cdot f(2 \cdot k + 1) + x \cdot f(2 \cdot k + 2)\}$
  $\{0 \le k < V \wedge Z = y \cdot f(2 \cdot k + 1) + x \cdot f(2 \cdot (k+1))\}$
  $(*$ definition $f(n)$; Note: $f(1) = 1 = 0 + 1 = f(0) + f(0+1) = f(2 \cdot 0 + 1)$ $*)$
  $\{0 \le k < V \wedge Z = y \cdot (f(k) + f(k+1)) + x \cdot f(k+1)\}$
  $\{0 \le k < V \wedge Z = y \cdot f(k) + (x + y) \cdot f(k+1)\}$
 $x := x + y$;
  $\{0 \le k < V \wedge Z = y \cdot f(k) + x \cdot f(k+1)\}$
end; $(*$ collect branches $*)$
$\{0 \le k < V \wedge Z = y \cdot f(k) + x \cdot f(k+1)\}$
$n := k$;
$\{0 \le n < V \wedge Z = y \cdot f(n) + x \cdot f(n+1)\}$
$\{J \wedge \mathrm{vf} < V\}$

university of groningen

Dijkstra's Fusc function (continued)

5 Conclusion: We derived the following program:

var $n,\ x,\ y :\ \mathbb{Z}$;
  $\{P : n \ge 0 \wedge Z = f(n)\}$
$y := 1$;
$x := 0$;
  $\{J :\ n \ge 0 \wedge Z = y \cdot f(n) + x \cdot f(n+1)\}$
    $(*\ vf = n\ *)$
while $n \ne 0$ **do**
  var $k := n$ **div** 2;
  if n **mod** $2 = 0$ **then**
    $y := x + y$;
  else
    $x := x + y$;
  end;
  $n := k$;
end;
  $\{Q : x = Z\}$

university of groningen

Fusc (in Dafny)

$$\begin{aligned} f(0) = 0 \quad &\wedge \quad f(1) = 1 \\ f(2 \cdot n) \quad &= \quad f(n) \\ f(2 \cdot n + 1) \quad &= \quad f(n) + f(n+1) \end{aligned}$$

We first need to rewrite the definition of the function `fusc`:

```
ghost function fusc(n: nat): nat {
  if n <= 1 then n
     else if n%2 == 0
          then fusc(n/2)
          else fusc(n/2) + fusc(1 + n/2)
}
```

university of groningen

Fusc (in Dafny)

```
method Fusc(m: nat) returns (x: nat)
ensures  x == fusc(m)
{
  var n := m;
  var y := 1;
  x := 0;
  while n != 0
  invariant 0<=n && fusc(m)==y*fusc(n) + x*fusc(n+1)
  {
    var k := n/2;
    assert 2*k == n || 2*k + 1 == n;
    if 2*k == n
    {
      y := x + y;
    } else {
      x := x + y;
    }
    n := k;
  }
}
```

university of groningen


```
Dafny program verifier finished with 3 verified, 0 errors
```

Fibonacci (once more)

The Fibonacci numbers are defined by the recurrence relation:

$$\begin{array}{lcl} f(0) & = & 0 \\ f(1) & = & 1 \\ f(n) & = & f(n-2)+f(n-1) \quad \text{for } n>1 \end{array}$$

We aim for a command S that satisfies: $\{P : n \geq 0\}\ S\ \{Q : x = f(n)\}$

It seems natural to aim for a loop in which a variable k runs from 0 to n which keeps $x = f(k) \wedge y = f(k-1)$ invariant.
In that case $f(k+1) = f(k-1) + f(k) = y + x$.

However, this is difficult to initialize.
If we initialize with $k := 0$ then we need $f(-1)$, which is undefined.

A better choice for the invariant is $J :\ 0 \leq k \leq n\ \wedge\ x = f(k)\ \wedge\ y = f(k+1)$.

Clearly, Q holds if $J\ \wedge\ k = n$, so we choose $B : k < n$ (or $k \neq n$).

Next, we choose a variant function.
The guard suggests to choose vf $= n - k \in \mathbb{Z}$.
The requirement vf ≥ 0 follows trivially from B.

university of groningen

Fibonacci (once more)

$$J :\ 0 \leq k \leq n\ \wedge\ x = f(k)\ \wedge\ y = f(k+1)$$

Initialization is easy: $k := 0;\ x := 0;\ y := 1;$

$$\begin{array}{lcl} & & \mathcal{WP}(k := 0; x := 0; y := 1,\ J) \\ \equiv & & 0 \leq 0 \leq n\ \wedge\ 0 = f(0)\ \wedge\ 1 = f(0+1) \\ \Leftarrow & & P \end{array}$$

For the body of the loop we find:

$\{0 \leq k < n\ \wedge\ x = f(k)\ \wedge\ y = f(k+1)\ \wedge\ n-k = V\}$
$\quad (*\ \text{prepare } k := k+1\ *)$
$\{0 < k+1 \leq n\ \wedge\ x = f(k+1-1)\ \wedge\ y = f(k+1)\ \wedge\ n-(k+1) < V\}$
$k := k+1;$
$\{0 < k \leq n\ \wedge\ x = f(k-1)\ \wedge\ y = f(k)\ \wedge\ n-k < V\}$
$\quad (*\ k > 0 \Rightarrow f(k+1) = f(k-1) + f(k) = x + y\ *)$
$\{0 < k \leq n\ \wedge\ x = f(k-1)\ \wedge\ x+y = f(k+1)\ \wedge\ n-k < V\}$
$y := x + y;$
$\{0 < k \leq n\ \wedge\ x = f(k-1)\ \wedge\ y = f(k+1)\ \wedge\ n-k < V\}$
$\quad (*\ k > 0 \Rightarrow f(k) = f(k+1) - f(k-1) = y - x\ *)$
$\{0 \leq k \leq n\ \wedge\ y-x = f(k)\ \wedge\ y = f(k+1)\ \wedge\ n-k < V\}$
$x := y - x;$
$\{0 \leq k \leq n\ \wedge\ x = f(k)\ \wedge\ y = f(k+1)\ \wedge\ n-k < V\}$

university of groningen

Fibonacci (Continued)

In conclusion, we derived the following program fragment:

const $n : \mathbb{Z}$;
var $k, x, y : \mathbb{Z}$;
  $\{P : n \geq 0\}$
$k := 0; x := 0; y := 1;$
  $\{J : 0 \leq k \leq n \wedge x = f(k) \wedge y = f(k+1)\}$
    $(*\ \mathrm{vf} = n - k\ *)$
while $k < n$ **do**
  $k := k + 1;$
  $y := x + y;$
  $x := y - x;$
end;
  $\{Q : x = f(n)\}$

university of groningen

Fibonacci (in Dafny)

```
function fib(n: nat): nat {
  if n <= 1 then n else fib(n-1) + fib(n-2)
}

method Fib(n: nat) returns (x: nat)
ensures  x == fib(n)
{
  var k,y;
  k,x,y := 0,0,1;
  while k < n
  invariant 0<=k<=n && x==fib(k) && y==fib(k+1)
  {
    k := k + 1;
    y := x + y;
    x := y - x;
  }
}
```

Dafny reports

```
Dafny program verifier finished with 3 verified, 0 errors
```

Lemmas and Proofs

So far, most programs were accepted by Dafny without much effort.
However, for more complicated programs the verifier needs our help.

This can be done by writing *lemmas*, which you can use in a larger proof.

Consider the following function, which returns something larger than its argument:

```
function More(x: int): int {
  if x <= 0 then 1 else More(x - 2) + 3
}
```

Next, we introduce the lemma `Increasing`:

```
lemma Increasing(x: int)
ensures x < More(x)
{
}
```

The body of a lemma may help Dafny to complete the proof. In this particular case, Dafny is able to prove the lemma itself, so we can leave the body empty.

university of groningen

Lemmas and Proofs

Next, consider the following `method UseLemma`:

```
method UseLemma(a: int) {
  var b := More(a);
  var c := More(b);
  assert 2 <= c - a;
}
```

Clearly, the method computes `More(More(a))` into variable `c`.
Since `More(a) >= a + 1`, we must surely have `More(More(a)) >= a+2`.

Still, Dafny fails to verify the program.
The reason is that the verifier does know the definition of `More` but not its properties.

In particular, it needs the `Increasing` lemma:

```
method UseLemma(a: int) {
  var b:= More(a);
  Increasing(a);
  Increasing(b);
  var c := More(b);
  assert 2 <= c - a;
}
```

university of groningen

Lemmas and Proofs

Why did the verifier accept the latter version? Let us zoom in:

```
function More(x: int): int {
  if x <= 0 then 1 else More(x - 2) + 3
}

lemma Increasing(x: int)
ensures x < More(x)
{
}

method UseLemma(a: int) {
  var b:= More(a);  // Dafny knows that b == More(a)
  Increasing(a);    // Dafny knows that More(a) > a, so b > a
  Increasing(b);    // Dafny knows that More(b) > b > a
  var c := More(b); // Dafny knows that c == More(b) > b > a
  assert 2 <= c - a; // c > b > a, so c - a >= 2
}
```

university of groningen

Proving a lemma

Dafny was able to prove the `Increasing` lemma, because it uses *automatic induction*.

You can turn off automatic induction:

```
function More(x: int): int {
  if x <= 0 then 1 else More(x - 2) + 3
}

lemma {:induction false} Increasing(x: int)
ensures x < More(x)
{
}
```

Dafny reports

```
Error:  a postcondition could not be proved on this return path
```

university of groningen

Proving a lemma

We need to help the verifier. We start with the following attempt:

```
lemma {:induction false} Increasing(x: int)
ensures x < More(x)
{
    if x <= 0 {
      // this case verifies without help (base case)
    } else {
      // this case needs our help (inductive case)
    }
}
```

Dafny reports

```
line 7:  a postcondition could not be proved on this return path
```

university of groningen

Proving a lemma

We need to help the verifier. We start with the following attempt:

```
lemma {:induction false} Increasing(x: int)
ensures x < More(x)
{
    if x <= 0 {
      // this case verifies without help (base case)
    } else {
      // x > 0  implies More(x) == More(x - 2) + 3
      // so x < More(x) is the same as x - 3 < More(x - 2)
      // that follows clearly from x - 2 < More(x - 2)
      Increasing(x-2);
    }
}
```

Dafny reports

```
Dafny program verifier finished with 0 errors
```

This completes the proof, but we need to realize that Dafny still took care of a detail.

Dafny used the `decreases x` clause to prove termination.

university of groningen

Proving a lemma (termination)

Proving termination is important, because it prevents proof attempts like this one:

```
lemma {:induction false} Increasing(x: int)
ensures x < More(x)
{
  Increasing(x); // this gives us x < More(x)
}
```

Dafny reports

```
cannot prove termination; try supplying a decreases clause
```

university of groningen

Proving a lemma (termination)

In Dafny, we can 'program' a proof:

```
lemma {:induction false} Increasing(x: int)
ensures x < More(x)
{
   if x > 0 {
     var y := x - 2;
     Increasing(y);
   }
}
```

Dafny reports

```
Dafny program verifier finished with 0 errors
```

university of groningen

Proof calculations

Dafny also supports calculations within a proof. They look like this:

```
calc {
    5*(x + 3);   // note the semicolon!
 == // distribute * over +
    5*x + 5*3;
 == // calculus
    5*x + 15;
}
```

Here is another example (`n` is of type `nat`):

```
calc {
    3*x + n + n;
 == // n + n == 2*n
    3*x + 2*n;
 <= // 2*n <= 3*n, since n >= 0
    3*x + 3*n;
  == // distribute * over +
    3*(x + n);
}
```

university of groningen

Proof calculations

A proof using `calc` looks like this:

```
lemma {:induction false} Increasing(x: int)
ensures x < More(x)
{
  if x <= 0 {
    calc {
         x;
      <= // guard says x <= 0
         0;
      <  // arithmetic
         1;
      == // definition More for x <= 0
         More(x);
     }
  } else {
    calc {
         More(x);
      == // definition More for x > 0
         More(x - 2) + 3;
      >  { Increasing(x-2); }   // hints to Dafny in curly braces
         x - 2 + 3;
      >  // calculus
         x;
     }
  }
}
```

university of groningen

Proof calculations

Here is a variation on the proof:

```
lemma {:induction false} Increasing(x: int)
ensures x < More(x)
{
  if x <= 0 {
    calc {
         x;
      <= // guard says x <= 0
         0;
      <  // arithmetic
         1;
      == // definition More for x <= 0
         More(x);
    }
  } else {
     calc {
        x < More(x);
     <== // calculus
        x + 1 < More(x);
     ==  // definition More for x > 0
        x + 1 < More(x - 2) + 3;
     ==  // calculus
        x - 2 < More(x - 2);
     <==  Increasing(x - 2);
        true;
     }
 }
```

university of groningen

Fibonacci squared (Dafny)

```
function fib(n: nat): nat {
  if n <= 1 then n else fib(n-1) + fib(n-2)
}
```

We aim for the following method:

```
method SquaredFib(N: nat) returns (x: nat)
ensures  x == fib(N)*fib(N)
{
  x := 0;
  var n := 0;
  while n != N
  invariant 0<=n<=N && x == fib(n)*fib(n)
  {
    ??????
    assert x == fib(n+1)*fib(n+1);
    n := n + 1;
    assert x == fib(n)*fib(n);
  }
}
```

Let us *wish* that we have `fib(n+1)*fib(n+1)` already computed in a variable `y`.

university of groningen

Fibonacci squared (Dafny)

```
method SquaredFib(N: nat) returns (x: nat)
ensures  x == fib(N)*fib(N)
{
  x := 0;
  var n := 0;
  while n != N
  invariant 0<=n<=N
  invariant x == fib(n)*fib(n)
  invariant y == fib(n + 1)*fib(n + 1)
  {
    x,y := y, ??;
    assert x == fib(n+1)*fib(n+1) && y == fib(n + 2)*fib(n + 2)
    n := n + 1;
    assert x == fib(n)*fib(n) && y == fib(n + 1)*fib(n + 1)
  }
}
```

university of groningen

Fibonacci squared (Dafny)

It is time for some calculation:

```
  while n != N
  invariant 0<=n<=N
  invariant x == fib(n)*fib(n)
  invariant y == fib(n + 1)*fib(n + 1)
  {
    calc {
      fib(n+2)*fib(n+2);
    == // definition fib
      (fib(n)+fib(n+1))*(fib(n)+fib(n+1));
    == // calculus
      fib(n)*fib(n) + 2*fib(n)*fib(n+1) + fib(n+1)*fib(n+1);
    == // invariant
      x + 2*fib(n)*fib(n+1) + y;
    }
    x,y := y, ??;
    assert x == fib(n+1)*fib(n+1) && y == fib(n + 2)*fib(n + 2)
    n := n + 1;
    assert x == fib(n)*fib(n) && y == fib(n + 1)*fib(n + 1)
  }
```

university of groningen

Ok, we have another *wish*: `k==2*fib(n)*fib(n + 1)`.

Fibonacci squared (Dafny)

```
  while n != N
  invariant 0<=n<=N
  invariant x == fib(n)*fib(n)
  invariant y == fib(n + 1)*fib(n + 1)
  invariant k == 2*fib(n)*fib(n + 1)
  {
    calc {
      2*fib(n+1)*fib(n+2);
    == // definition fib
      2*fib(n+1)*(fib(n) + fib(n+1));
    == // calculus
      2*fib(n+1)*fib(n) + 2*fib(n+1)*fib(n+1);
    == // invariant
      k + 2*y;
    }
    calc {
      fib(n+2)*fib(n+2);
    == // definition fib
      (fib(n)+fib(n+1))*(fib(n)+fib(n+1));
    == // calculus
      fib(n)*fib(n) + 2*fib(n)*fib(n+1) + fib(n+1)*fib(n+1);
    == // invariant
      x + k + y;
    }
    x,y,k := y, x + k + y, k + 2*y;
    n := n + 1;
  }
```

university of groningen

Fibonacci squared (Dafny)

In conclusion, we have found the following method implementation:

```
method SquaredFib(N: nat) returns (x: nat)
ensures  x == fib(N)*fib(N)
{
  x := 0;
  var n, y, k := 0, 1, 0;
  while n != N
  invariant 0<=n<=N
  invariant x == fib(n)*fib(n)
  invariant y == fib(n + 1)*fib(n + 1)
  invariant k == 2*fib(n)*fib(n + 1)
  {
    x,y,k := y, x + k + y, k + 2*y;
    n := n + 1;
  }
}
```

university of groningen

Notation

Before we continue, we need some extra notation.

This notation lacks in the book, but it makes manual proofs easier.

university of groningen

Notation: logical quantifiers

- equivalence ($\equiv$): $P \equiv Q$ is notation for $(P \Rightarrow Q) \wedge (Q \Rightarrow P)$
- $\forall x \in V : P(x) \quad \equiv \quad \forall x(x \in V \Rightarrow P(x))$
- $\exists x \in V : P(x) \quad \equiv \quad \exists x(x \in V \ \wedge \ P(x))$

university of groningen

Notation: Sets

- standard notation: $\{x \mid P(x)\}$
  For example $\{x \mid 0 \leq x^3 \leq 100\} = \{0, 1, 2, 3, 4\}$
- alternative notation: $\{E(x) \mid x : P(x)\}$, e.g.
  $\{2 \cdot x \mid x : 0 \leq x \leq 5\} = \{0, 2, 4, 6, 8, 10\}$
- notations are equivalent: $\{E(x) \mid x : P(x)\} = \{y \mid \exists x(y = E(x) \wedge P(x))\}$
- empty set: $\emptyset$
- element: $w \in V$ means "w is member of the set V"
- $\#V$: number of elements of the set V
- $\#\emptyset = 0$
- singleton rule: $\#\{a\} = 1$
- split rule: $\#V = \#(V \cap W) + \#(V \backslash W)$
- ord(**true**) $= 1$ and ord(**false**) $= 0$
- $\#V = \#(V \backslash \{a\}) + \text{ord}(a \in V)$

university of groningen

Notation: Segments

- Segment/interval: consecutive subset of $\mathbb{Z}$
- closed: $[m..n] = \{k \mid m \leq k \leq n\}$
- open: $(m..n) = \{k \mid m < k < n\}$
- half open: $[m..n) = \{k \mid m \leq k < n\}$
- half open: $(m..n] = \{k \mid m < k \leq n\}$
- Advice: use half open intervals!
  Why? Well, $\#[m..n) = \#(m..n] = n - m$

university of groningen

Notation: Maxima/Minima Sets/Segments

- Max V: maximum element of the set V
- Min V: minimum element of the set V
- For $V \neq \emptyset$ we have
  $a = \text{Max } V \quad \equiv \quad a \in V \ \wedge \ \forall x(x \in V \Rightarrow x \leq a)$
  $a = \text{Min } V \quad \equiv \quad a \in V \ \wedge \ \forall x(x \in V \Rightarrow a \leq x)$
- operator **max**: x **max** $y = \text{Max } \{x, y\}$
- operator **min**: x **min** $y = \text{Min } \{x, y\}$
- singleton rules: $\text{Max } \{a\} = a$ and $\text{Min } \{a\} = a$
- empty set rules: $\text{Max } \emptyset = -\infty$ and $\text{Min } \emptyset = \infty$
- split rule: $\text{Max } V = \text{Max } (V \cap W)$ **max** $\text{Max } (V \setminus W)$
- split rule: $\text{Min } V = \text{Min } (V \cap W)$ **min** $\text{Min } (V \setminus W)$

university of groningen

Notation: Families/Bags/Multi sets

- $(f(i) \mid i \in V)$ is the *family* consisting of the values $f(i)$ for all $i \in V$.
- Alternative notation: $(f(i) \mid i \in V : P(i))$
- Note that the (above) family differs from the set $\{f(i) \mid i \in V\}$
- $\{i \textbf{ div } 2 \mid i \in [0..5)\} = \{0, 1, 2\}$
- $(i \textbf{ div } 2 \mid i \in [0..5)) = (0, 0, 1, 1, 2)$
- In a family, the order of elements is relevant, so
  $(1, 0, 0) \neq (0, 1, 0)$, $(1, 0, 0) \neq (0, 0, 1)$, and $(0, 1, 0) \neq (0, 0, 1)$.

university of groningen

Notation: Sum operator on families

- $\Sigma(f(i) \mid i \in V)$ is the sum of the elements of the *family*.
- $\Sigma(f(i) \mid i \in \emptyset) = 0$
- $\Sigma(f(i) \mid i \in \{a\}) = f(a)$
- $\Sigma(f(i) \mid i \in V) = \Sigma(f(i) \mid i \in V \cap W) + \Sigma(f(i) \mid i \in V \setminus W)$

university of groningen

Notation: Product operator on families

- $\prod(f(i) \mid i \in V)$ is the product of the elements of the *family*.
- $\prod(f(i) \mid i \in \emptyset) = 1$
- $\prod(f(i) \mid i \in \{a\}) = f(a)$
- $\prod(f(i) \mid i \in V) = \prod(f(i) \mid i \in V \cap W) \cdot \prod(f(i) \mid i \in V \setminus W)$

university of groningen

Notation: Distributive laws

- mutiplication over addition: $\Sigma(x \cdot f(i) \mid i \in V) = x \cdot \Sigma(f(i) \mid i \in V)$
- addition over Max : $\text{Max}\ (x + f(i) \mid i \in V) = x + \text{Max}\ (f(i) \mid i \in V)$
- addition over Min : $\text{Min}\ (x + f(i) \mid i \in V) = x + \text{Min}\ (f(i) \mid i \in V)$
- multiplication over Max :
  $x \geq 0 \ \wedge \ V \neq \emptyset \Rightarrow \text{Max}\ (x \cdot f(i) \mid i \in V) = x \cdot \text{Max}\ (f(i) \mid i \in V)$
- multiplication over Min :
  $x \geq 0 \ \wedge \ V \neq \emptyset \Rightarrow \text{Min}\ (x \cdot f(i) \mid i \in V) = x \cdot \text{Min}\ (f(i) \mid i \in V)$

But,

- $x < 0 \ \wedge \ V \neq \emptyset \Rightarrow \text{Max}\ (x \cdot f(i) \mid i \in V) = x \cdot \text{Min}\ (f(i) \mid i \in V)$
- $x < 0 \ \wedge \ V \neq \emptyset \Rightarrow \text{Min}\ (x \cdot f(i) \mid i \in V) = x \cdot \text{Max}\ (f(i) \mid i \in V)$

university of groningen

Example: Summing an array

const $n : \mathbb{N},\ a :$ **array** $[0..n)$ **of** $\mathbb{Z}$;
var $x : \mathbb{Z}$;
$\{P :$ **true** $\}$
S
$\{Q : x = \Sigma(a[i] \mid i : i \in [0..n))\}$

To reduce the size of our formulas we introduce (for $0 \leq k \leq n$):

$$S(k) = \Sigma(a[i] \mid i : i \in [0..k))$$

Next, we rewrite the postcondition: $Q : x = S(n)$

university of groningen

Example: $S(k) = \Sigma(a[i] \mid i : i \in [0..k))$

It is clear that $S(0) = 0$, since the domain of the sum is empty.

For $0 \leq k < n$, we compute $S(k+1)$:

$$
\begin{array}{ll}
& S(k+1) \\
= & \{ \text{ definition } \} \\
& \Sigma(a[i] \mid i : i \in [0..k+1)) \\
= & \{ \text{ split domain: } i = k \ \vee \ i < k \ \} \\
& a[k] + \Sigma(a[i] \mid i : i \in [0..k)) \\
= & \{ \text{ definition } \} \\
& a[k] + S(k)
\end{array}
$$

So, we found the following recurrence:

$$
\begin{array}{rcl}
 & & S(0) = 0 \\
0 \leq k < n & \Rightarrow & S(k+1) = a[k] + S(k)
\end{array}
$$

university of groningen

Summing an array: step-by-step

1. Choose an invariant J, and guard B.
   The postcondition is $Q : \ x = S(n)$.
   We use the heuristic *replace the constant n by a variable k* and augment the invariant with the domain condition $0 \leq k \leq n$.

$$
\begin{array}{c}
J : \ 0 \leq k \leq n \ \wedge \ x = S(k) \\
B : \ k \neq n
\end{array}
$$

$$
\begin{array}{ll}
& J \ \wedge \ \neg B \\
\equiv & \{ \text{ definition } J \text{ and } B \} \\
& 0 \leq k \leq n \ \wedge \ x = S(k) \ \wedge \ k = n \\
\Rightarrow & \{ \text{ substitute } k = n; \text{ logic } \} \\
& Q : x = S(n)
\end{array}
$$

university of groningen

Summing an array: step-by-step

2 Initialization: Find a command T_0 such that
$\{P:\ \textbf{true}\}\ T_0\ \{J:\ 0 \le k \le n\ \wedge\ x = S(k)\}$

This is easy: since $S(0) = 0$ we choose $k := 0;\ x := 0;$

$\{P:\ \textbf{true}\}$
$\{0 \le 0 \le n\ \wedge\ 0 = S(0)\}$
$k, x := 0, 0;$
$\{J:\ 0 \le k \le n\ \wedge\ x = S(k)\}$

3 Variant function: choose a vf $\in \mathbb{Z}$ and prove $J\ \wedge\ B \Rightarrow \text{vf} \ge 0$.
Since initialliy $k = 0$ an the guard is $k \neq n$, we need to increase k.
So, we choose $\text{vf} = n - k \in \mathbb{Z}$. Clearly, $J\ \wedge\ B \Rightarrow J \Rightarrow k \le n \equiv vf \ge 0$

4 Body of the loop: $\{J\ \wedge\ B\ \wedge\ \text{vf} = V\}\ S\ \{J\ \wedge\ \text{vf} < V\}$.

$\{0 \le k \le n\ \wedge\ x = S(k)\ \wedge\ k \neq n\ \wedge\ n - k = V\}$
$(*$ calculus; $k < n$; prepare $k := k + 1$; use recurrence $*)$
$\{0 \le k + 1 \le n\ \wedge\ x + a[k] = S(k + 1)\ \wedge\ n - (k + 1) < V\}$
$x := x + a[k];$
$\{0 \le k + 1 \le n\ \wedge\ x = S(k + 1)\ \wedge\ n - (k + 1) < V\}$
$k := k + 1;$
$\{0 \le k \le n\ \wedge\ x = S(k)\ \wedge\ n - k < V\}$

university of groningen

Summing an array: step-by-step

5 Conclusion: we derived the following program fragment:

const $n:\ \mathbb{N},\ a:$ **array** $[0..n)$ **of** $\mathbb{Z}$;
var $x:\ \mathbb{Z}$;
$\{P:\ \textbf{true}\}$
$k := 0;$
$x := 0;$
$\{J: 0 \le k \le n\ \wedge\ x = S(k)\}$
$(*\ \text{vf} = n - k\ *)$
while $k \neq n$ **do**
$x := x + a[k];$
$k := k + 1;$
end;
$\{Q:\ x = \Sigma(a[i] \mid i : i \in [0..n))\}$

university of groningen

Summing an array (Dafny version)

```
ghost function sum(a: array<int>, upb: int) : int
requires 0 <= upb <= a.Length
reads a
{
    if upb == 0 then 0
    else a[upb - 1] + sum(a, upb - 1)
}


method sumArray(a: array<int>) returns (x: int)
ensures x == sum(a, a.Length)
{
    x := 0;
    var k: int := 0;
    while k < a.Length
        invariant 0 <= k <= a.Length && x == sum(a, k)
        decreases a.Length - k
    {
        x := x + a[k];
        k := k + 1;
    }
}
```

university of groningen

Example: a challenging sum

We aim for a command T that satisfies the specification:

const $n : \mathbb{N}$, $a :$ **array** $[0..n)$ **of** $\mathbb{Z}$;
var $s : \mathbb{Z}$;
$\{P : n \geq 1\}$
T
$\{Q : s = \Sigma(a[i] \cdot a[j] \cdot a[k] \mid i, j, k : 0 \leq i < j \leq k < n)\}$

university of groningen

Example: a challenging sum

Clearly, the following program fragment satisfies the specification:

const $n : \mathbb{N},\ a :$ **array** $[0..n)$ **of** $\mathbb{Z}$;
var $i, j, k, s : \mathbb{Z}$;
$\{P :\ n \geq 1\}$
$s, i\ :=\ 0, 0$;
while $i < n - 1$ **do**
  $j := i + 1$;
  while $j < n$ **do**
    $k := j$;
    while $k < n$ **do**
      $s := s + a[i] * a[j] * a[k]$;
      $k := k + 1$;
    end;
    $j := j + 1$;
  end;
  $i := i + 1$;
end;
$\{Q :\ s = \Sigma(a[i] \cdot a[j] \cdot a[k] \mid i, j, k : 0 \leq i < j \leq k < n)\}$

But, it needs about n^3 steps.
We can do much better with a precise derivation!

university of groningen

A challenging sum (continued)

$$s = \Sigma(a[i] \cdot a[j] \cdot a[k] \mid i, j, k : 0 \leq i < j \leq k < n)$$

We start by introducing

$$S(x) = \Sigma(a[i] \cdot a[j] \cdot a[k] \mid i, j, k : 0 \leq i < j \leq k < x)$$

and rewrite the postcondition: $Q :\ s = S(n)$

1. Choose an invariant J, and guard B.
   As a first attempt we introduce a variable x and try to maintain

   $$J :\ s = S(x)\ \wedge\ 1 \leq x \leq n$$

   Clearly, we choose $B :\ x \neq n$, such that $J\ \wedge\ \neg B \Rightarrow Q$.

Note: during the derivation we can safely add conjuncts to J (it makes J stronger):

$$(J' \Rightarrow J)\ \wedge\ (J\ \wedge\ \neg B \Rightarrow Q) \quad \Rightarrow \quad (J'\ \wedge\ \neg B \Rightarrow Q)$$

university of groningen

A challenging sum (continued)

$$S(x) = \Sigma(a[i] \cdot a[j] \cdot a[k] \mid i, j, k : 0 \leq i < j \leq k < x)$$

Clearly, the idea is to use a loop in which x is incremented. Therefore, we first have a look at $S(x+1)$.

$$\begin{array}{ll} & S(x+1) \\ = & \quad \{\text{definition } S\} \\ & \Sigma(a[i] \cdot a[j] \cdot a[k] \mid i, j, k : 0 \leq i < j \leq k < x+1) \\ = & \quad \{\textbf{assume } x \geq 1; \text{ split domain: } k < x \ \vee \ k = x\} \\ & \Sigma(a[i] \cdot a[j] \cdot a[k] \mid i, j, k : 0 \leq i < j \leq k < x) \ + \Sigma(a[i] \cdot a[j] \cdot a[x] \mid i, j : 0 \leq i < j \leq x) \\ = & \quad \{\text{definition } S \text{ and calculus}\} \\ & S(x) + a[x] \cdot \Sigma(a[i] \cdot a[j] \mid i, j : 0 \leq i < j \leq x) \\ = & \quad \{\text{we prefer half-open intervals}\} \\ & S(x) + a[x] \cdot \Sigma(a[i] \cdot a[j] \mid i, j : 0 \leq i < j < x+1) \\ = & \quad \{\text{introduce } T\} \\ & S(x) + a[x] \cdot T(x+1) \quad \text{where } T(x) = \Sigma(a[i] \cdot a[j] \mid i, j : 0 \leq i < j < x) \end{array}$$

university of groningen

A challenging sum (continued)

So, if we want to increment x, we need $T(x+1)$.

$$\begin{array}{ll} & T(x+1) \\ = & \quad \{\text{definition } T\} \\ & \Sigma(a[i] \cdot a[j] \mid i, j : 0 \leq i < j < x+1) \\ = & \quad \{\textbf{assume } x \geq 1; \text{ split domain: } j < x \ \vee \ j = x\} \\ & \Sigma(a[i] \cdot a[j] \mid i, j : 0 \leq i < j < x) \ + \ \Sigma(a[i] \cdot a[x] \mid i : 0 \leq i < x) \\ = & \quad \{\text{definition } T; \text{ calculus}\} \\ & T(x) \ + \ a[x] \cdot \Sigma(a[i] \mid i : 0 \leq i < x) \\ = & \quad \{\text{introduce } U\} \\ & T(x) + a[x] \cdot U(x) \quad \text{where } U(x) = \Sigma(a[i] \mid i : 0 \leq i < x) \end{array}$$

We also have a look at $U(x+1)$.

$$\begin{array}{ll} & U(x+1) \\ = & \quad \{\text{definition } U\} \\ & \Sigma(a[i] \mid i : 0 \leq i < x+1) \\ = & \quad \{\textbf{assume } x \geq 0; \text{ split domain: } i < x \ \vee \ i = x\} \\ & \Sigma(a[i] \mid i : 0 \leq i < x) \ + \ a[x] \\ = & \quad \{\text{definition } U\} \\ & U(x) \ + \ a[x] \end{array}$$

university of groningen

A challenging sum (continued)

For the following definitions

$$\begin{array}{lcl} S(x) & = & \Sigma(a[i] \cdot a[j] \cdot a[k] \mid 0 \leq i < j \leq k < x) \\ T(x) & = & \Sigma(a[i] \cdot a[j] \mid i, j : 0 \leq i < j < x) \\ U(x) & = & \Sigma(a[i] \mid i : 0 \leq i < x) \end{array}$$

we found the recurrences

$$\begin{array}{rcl} x \geq 1 & \Rightarrow & S(x+1) = S(x) + a[x] \cdot T(x+1) \\ x \geq 1 & \Rightarrow & T(x+1) = T(x) + a[x] \cdot U(x) \\ x \geq 0 & \Rightarrow & U(x+1) = a[x] + U(x) \end{array}$$

$$\begin{array}{rcl} S(x) = 0 & \Leftarrow & x \leq 1 \\ T(x) = 0 & \Leftarrow & x \leq 1 \\ U(x) = 0 & \Leftarrow & x \leq 0 \end{array}$$

university of groningen

A challenging sum (continued)

$$\begin{array}{lcl} S(x) & = & \Sigma(a[i] \cdot a[j] \cdot a[k] \mid 0 \leq i < j \leq k < x) \\ T(x) & = & \Sigma(a[i] \cdot a[j] \mid i, j : 0 \leq i < j < x) \\ U(x) & = & \Sigma(a[i] \mid i : 0 \leq i < x) \end{array}$$

2 Initialization: We now strengthen the invariant and initialize it.

$$J : \ s = S(x) \ \wedge \ t = T(x) \ \wedge \ u = U(x) \ \wedge \ 1 \leq x \leq n$$

$\{P : \ n \geq 1\}$
  $(*$ base cases recurrences; $U(1) = a[0] + U(0) = a[0]$ $*)$
$\{0 = S(1) \ \wedge \ 0 = T(1) \ \wedge \ a[0] = U(1) \ \wedge \ 1 \leq 1 \leq n\}$
$s, t, u, x := 0, 0, a[0], 1;$
$\{J : \ s = S(x) \ \wedge \ t = T(x) \ \wedge \ u = U(x) \ \wedge \ 1 \leq x \leq n\}$

3 Variant function: vf $= n - x \in \mathbb{Z}$. Clearly, $J \ \wedge \ B \Rightarrow$ vf ≥ 0

university of groningen

A challenging sum (continued)

4 Body: $\{J \wedge B \wedge \text{vf} = V\}\ S\ \{J \wedge \text{vf} < V\}$

$\{s = S(x) \wedge t = T(x) \wedge u = U(x) \wedge 1 \leq x < n \wedge n - x = V\}$
$(*$ recurrence $T(x+1) = T(x) + a[x] \cdot U(x)$; substitution; logic $*)$
$\{s = S(x) \wedge t + a[x] \cdot u = T(x+1) \wedge u = U(x) \wedge 1 \leq x < n \wedge n - x = V\}$
$t := t + a[x] * u;$
$\{s = S(x) \wedge t = T(x+1) \wedge u = U(x) \wedge 1 \leq x < n \wedge n - x = V\}$
$(*$ recurrence $S(x+1) = S(x) + a[x] \cdot T(x+1)$; substitution $*)$
$\{s + a[x] \cdot t = S(x+1) \wedge t = T(x+1) \wedge u = U(x) \wedge 1 \leq x < n \wedge n - x = V\}$
$s := s + a[x] * t;$
$\{s = S(x+1) \wedge t = T(x+1) \wedge u = U(x) \wedge 1 \leq x < n \wedge n - x = V\}$
$(*$ recurrence $U(x+1) = U(x) + a[x]$ $*)$
$\{s = S(x+1) \wedge t = T(x+1) \wedge u + a[x] = U(x+1) \wedge 1 \leq x < n \wedge n - x = V\}$
$u := u + a[x];$
$\{s = S(x+1) \wedge t = T(x+1) \wedge u = U(x+1) \wedge 1 \leq x < n \wedge n - x = V\}$
$(*$ prepare $x := x + 1$ $*)$
$\{s = S(x+1) \wedge t = T(x+1) \wedge u = U(x+1) \wedge 1 \leq x+1 \leq n \wedge n - (x+1) < V\}$
$x := x + 1;$
$\{s = S(x) \wedge t = T(x) \wedge u = U(x) \wedge 1 \leq x \leq n \wedge n - x < V\}$
$(*$ definitions J, and vf $*)$
$\{J \wedge \text{vf} < V\}$

university of groningen

A challenging sum (continued)

5 Conclusion: We arrived at the following program fragment:

const $n : \mathbb{N},\ a :$ **array** $[0..n)$ **of** $\mathbb{Z}$;
var $x, s, t, u : \mathbb{Z}$;
$\{P :\ n \geq 1\}$
$s, t, u, x := 0, 0, a[0], 1;$
$\{J :\ s = S(x) \wedge t = T(x) \wedge u = U(x) \wedge 1 \leq x \leq n\}$
$(*$ vf $= n - x$ $*)$
while $x < n$ **do**
$t := t + a[x] * u;$
$s := s + a[x] * t;$
$u := u + a[x];$
$x := x + 1;$
end;
$\{Q :\ s = \Sigma(a[i] \cdot a[j] \cdot a[k] \mid i, j, k : 0 \leq i < j \leq k < n)\}$

This program needs only n iterations, which is a major improvement over the original brute force algorithm.

university of groningen

A challenging sum (Dafny version)

```
ghost function U(upb: int, a: array<int>) : int
 requires 0 <= upb <= a.Length
 reads a
{
  if upb == 0 then 0
  else a[upb - 1] + U(upb - 1, a)
}

ghost function T(upb: int, a: array<int>) : int
 requires 0 <= upb <= a.Length
 reads a
{
  if upb == 0 then 0
  else T(upb - 1, a) + a[upb - 1]*U(upb - 1, a)
}

ghost function S(upb: int, a: array<int>) : int
 requires 0 <= upb <= a.Length
 reads a
{
  if upb == 0 then 0
  else S(upb - 1, a) + a[upb - 1]*T(upb, a)
}
```

university of groningen

A challenging sum (Dafny version)

```
method tripleSum(a: array<int>) returns (s: int)
 requires a.Length > 0
 ensures s == S(a.Length, a)
{
  var t,u,x;
  s,t,u,x := 0,0,a[0],1;
  while x < a.Length
  invariant 0 <= x <= a.Length
  invariant t == T(x, a) && u == U(x, a) && s == S(x, a)
  {
    t := t + u*a[x];
    s := s + t*a[x];
    u := u + a[x];
    x := x + 1;
  }
}
```

university of groningen

Another summing example

Derive a command to compute

$$\Sigma(a[i] \cdot a[j] \cdot a[k] \mid i, j, k : 0 \leq i \leq j < n \ \wedge \ i \leq k < n)$$

We start by introducing (we replace the lower bound!):

$$S(x) = \Sigma(a[i] \cdot a[j] \cdot a[k] \mid i, j, k : x \leq i \leq j < n \ \wedge \ i \leq k < n)$$

So, we want to find a command T that satisfies the specification:

const $n : \ \mathbb{N}, \ a : \ $ **array** $[0..n)$ **of** $\mathbb{Z}$;
var $s : \ \mathbb{Z}$;
$\{P : \ n \geq 1\}$
T
$\{Q : \ s = S(0)\}$

university of groningen

Another summing example (continued)

$$S(x) = \Sigma(a[i] \cdot a[j] \cdot a[k] \mid i, j, k : x \leq i \leq j < n \ \wedge \ i \leq k < n)$$

1. Choose an invariant J, and guard B.
   As a first attempt we introduce a variable x and try to maintain

   $$J : \ s = S(x) \ \wedge \ 0 \leq x \leq n$$

   Clearly, we choose $B : \ x \neq 0$, such that $J \ \wedge \ \neg B \Rightarrow Q$.

university of groningen

Another summing example (continued)

$$S(x) = \Sigma(a[i] \cdot a[j] \cdot a[k] \mid i,j,k : x \leq i \leq j < n \ \wedge \ i \leq k < n)$$

Clearly, the idea is to use a loop in which x is decremented.
So, we have a look at $S(x-1)$, for $x > 0$.

$$\begin{array}{ll} & S(x-1) \\ = & \quad \{\text{definition } S\} \\ & \Sigma(a[i] \cdot a[j] \cdot a[k] \mid i,j,k : x-1 \leq i \leq j < n \ \wedge \ i \leq k < n) \\ = & \quad \{\text{split domain: } x \leq i \ \vee \ i = x-1\} \\ & \Sigma(a[i] \cdot a[j] \cdot a[k] \mid i,j,k : x \leq i \leq j < n \ \wedge \ i \leq k < n) + \\ & \Sigma(a[x-1] \cdot a[j] \cdot a[k] \mid j,k : x-1 \leq j < n \ \wedge \ x-1 \leq k < n) \\ = & \quad \{\text{definition } S\text{; calculus}\} \\ & S(x) + a[x-1] \cdot \Sigma(a[j] \cdot a[k] \mid j,k : x-1 \leq j < n \ \wedge \ x-1 \leq k < n) \\ = & \quad \{\text{introduce } T\} \\ & S(x) + a[x-1] \cdot T(x-1) \end{array}$$

where $T(x) = \Sigma(a[j] \cdot a[k] \mid j,k : x \leq j < n \ \wedge \ x \leq k < n)$

university of groningen

Another summing example (continued)

So, if we want to decrement x, we need $T(x-1)$ for $x > 0$:

$$\begin{array}{ll} & T(x-1) \\ = & \quad \{\text{definition } T\} \\ & \Sigma(a[j] \cdot a[k] \mid j,k : x-1 \leq j < n \ \wedge \ x-1 \leq k < n) \\ = & \quad \{\text{split domain: } x \leq j \ \vee \ j = x-1\} \\ & \Sigma(a[j] \cdot a[k] \mid j,k : x \leq j < n \ \wedge \ x-1 \leq k < n) + \\ & \Sigma(a[x-1] \cdot a[k] \mid k : x-1 \leq k < n) \\ = & \quad \{\text{split domain of 1st term: } x \leq k \ \vee \ k = x-1\} \\ & \Sigma(a[j] \cdot a[k] \mid j,k : x \leq j < n \ \wedge \ x \leq k < n) + \\ & \Sigma(a[j] \cdot a[x-1] \mid j : x \leq j < n) + \\ & \Sigma(a[x-1] \cdot a[k] \mid k : x-1 \leq k < n) \\ = & \quad \{\text{definition } T\text{; calculus}\} \\ & T(x) \ + \ a[x-1] \cdot (\Sigma(a[k] \mid k : x-1 \leq k < n) \ + \Sigma(a[j] \mid j : x \leq j < n)) \\ = & \quad \{\text{rename bound variable: } k \rightsquigarrow j\} \\ & T(x) \ + \ a[x-1] \cdot (\Sigma(a[j] \mid j : x-1 \leq j < n) \ + \Sigma(a[j] \mid j : x \leq j < n)) \\ = & \quad \{\text{introduce } U\} \\ & T(x) + a[x-1] \cdot (U(x-1) + U(x)) \quad \text{where } U(x) = \Sigma(a[i] \mid i : x \leq i < n) \end{array}$$

university of groningen

Another summing example (continued)

We also have a look at $U(x-1)$ for $x > 0$.

$$
\begin{array}{ll}
& U(x-1) \\
= & \quad \{\text{definition } U\} \\
& \Sigma(a[i] \mid i : x-1 \leq i < n) \\
= & \quad \{\text{split domain: } x \leq i \ \vee \ i = x-1\} \\
& \Sigma(a[i] \mid i : x \leq i < n) \ + \ a[x-1] \\
= & \quad \{\text{definition } U\} \\
& U(x) \ + \ a[x-1]
\end{array}
$$

Another summing example (continued)

For the following definitions

$$
\begin{array}{rcl}
S(x) & = & \Sigma(a[i] \cdot a[j] \cdot a[k] \mid i,j,k : x \leq i \leq j < n \ \wedge \ i \leq k < n) \\
T(x) & = & \Sigma(a[j] \cdot a[k] \mid j,k : x \leq j < n \ \wedge \ x \leq k < n) \\
U(x) & = & \Sigma(a[i] \mid i : x \leq i < n)
\end{array}
$$

we found the recurrences

$$
\begin{array}{lcl}
x = n & \Rightarrow & S(x) = T(x) = U(x) = 0 \\
x > 0 & \Rightarrow & S(x-1) = S(x) + a[x-1] \cdot T(x-1) \\
x > 0 & \Rightarrow & T(x-1) = T(x) + a[x-1] \cdot (U(x-1) + U(x)) \\
x > 0 & \Rightarrow & U(x-1) = a[x-1] + U(x)
\end{array}
$$

Another summing example (continued)

2 Initialization: We now strengthen the invariant and initialize it.

$$J: \ s = S(x) \ \wedge \ t = T(x) \ \wedge \ u = U(x) \ \wedge \ 0 \leq x \leq n$$

$\{P: \ n \geq 1\}$
$(*$ base cases recurrences; calculus $*)$
$\{0 = S(n) \ \wedge \ 0 = T(n) \ \wedge \ 0 = U(n) \ \wedge \ 0 \leq n \leq n\}$
$s, t, u, x := 0, 0, 0, n;$
$\{J: \ s = S(x) \ \wedge \ t = T(x) \ \wedge \ u = U(x) \ \wedge \ 0 \leq x \leq n\}$

3 Variant function: vf $= x \in \mathbb{Z}$. Clearly, $J \ \wedge \ B \Rightarrow$ vf ≥ 0

university of groningen

Another summing example (continued)

4 Body: $\{J \ \wedge \ B \ \wedge$ vf $= V\}$ S $\{J \ \wedge$ vf $< V\}$

$\{J \ \wedge \ B \ \wedge$ vf $= V\}$
$(*$ definitions J, B, and vf; logic $*)$
$\{s = S(x) \ \wedge \ t = T(x) \ \wedge \ u = U(x) \ \wedge \ 0 < x = V \leq n\}$
$(*$ recurrence $T(x-1) = T(x) + a[x-1] \cdot (U(x-1) + U(x))$; substitution $*)$
$\{s = S(x) \ \wedge \ t + a[x-1] \cdot (U(x-1) + u) = T(x-1) \wedge u = U(x) \wedge ....\}$
$(*$ calculus $*)$
$\{s = S(x) \ \wedge \ t + a[x-1] \cdot u + a[x-1] \cdot U(x-1) = T(x-1) \wedge ....\}$
$t := t + a[x-1] * u;$
$\{s = S(x) \ \wedge \ t + a[x-1] \cdot U(x-1) = T(x-1) \wedge u = U(x) \wedge 0 < x = V \leq n\}$
$(*$ recurrence $U(x-1) = U(x) + a[x-1] *)$
$\{s = S(x) \ \wedge \ t + a[x-1] \cdot U(x-1) = T(x-1) \wedge u + a[x-1] = U(x-1) \wedge ....\}$
$u := u + a[x-1];$
$\{s = S(x) \wedge t + a[x-1] \cdot U(x-1) = T(x-1) \wedge u = U(x-1) \wedge 0 < x = V \leq n\}$
$(*$ substitution $*)$
$\{s = S(x) \ \wedge \ t + a[x-1] \cdot u = T(x-1) \ \wedge \ u = U(x-1) \ \wedge \ 0 < x = V \leq n\}$
$t := t + a[x-1] * u;$
$\{s = S(x) \ \wedge \ t = T(x-1) \ \wedge \ u = U(x-1) \ \wedge \ 0 < x = V \leq n\}$

university of groningen

Another summing example (continued)

4 Body: continued

$\{s = S(x) \ \wedge \ t = T(x-1) \ \wedge \ u = U(x-1) \ \wedge \ 0 < x = V \leq n\}$
$(*$ recurrence $S(x-1) = S(x) + a[x-1] \cdot T(x-1)$; substitution $*)$
$\{s + a[x-1] \cdot t = S(x-1) \ \wedge \ t = T(x+1) \ \wedge \ u = U(x) \ \wedge \ 0 < x = V \leq n\}$
$s := s + a[x-1] * t;$
$\{s = S(x-1) \ \wedge \ t = T(x-1) \ \wedge \ u = U(x-1) \ \wedge \ 0 < x = V \leq n\}$
$(*$ calculus $*)$
$\{s = S(x-1) \ \wedge \ t = T(x-1) \ \wedge \ u = U(x-1) \ \wedge \ 0 \leq x-1 \leq n \ \wedge \ x-1 < V\}$
$x := x - 1;$
$\{s = S(x) \ \wedge \ t = T(x) \ \wedge \ u = U(x) \ \wedge \ 0 \leq x \leq n \ \wedge \ x < V\}$
$(*$ definitions J, and vf $*)$
$\{J \ \wedge \ \text{vf} < V\}$

university of groningen

Another summing example (continued)

5 Conclusion

We derived the program fragment:

const $n : \mathbb{N}, \ a :$ **array** $[0..n)$ **of** $\mathbb{Z}$;
var $x, s, t, u : \mathbb{Z}$;
$\{P : \ n \geq 1\}$
$s := 0; \ t := 0; \ u := 0; \ x := n;$
$\{J : \ s = S(x) \ \wedge \ t = T(x) \ \wedge \ u = U(x) \ \wedge \ 0 \leq x \leq n\}$
$(*$ vf $= x *)$
while $x \neq 0$ **do**
$t := t + a[x-1] * u;$
$u := u + a[x-1];$
$t := t + a[x-1] * u;$
$s := s + a[x-1] * t;$
$x := x - 1;$
end;
$\{Q : \ s = \Sigma(a[i] \cdot a[j] \cdot a[k] \mid i, j, k : 0 \leq i \leq j < n \ \wedge \ i \leq k < n)\}$

university of groningen

Another summing example (Dafny version)

```
method TripleSum(a: array<int>) returns (s: int)
 requires a.Length > 0
 ensures s == S(0, a)
{
  var t,u,x;
  s,t,u,x := 0,0,0,a.Length;
  while (x > 0)
   invariant 0 <= x <= a.Length
   invariant t == T(x, a) && u == U(x, a) && s == S(x, a)
  {
     t := t + a[x-1]*u;
     u := u + a[x-1];
     t := t + a[x-1]*u;
     s := s + a[x-1]*t;
     x := x - 1;
  }
}
```

university of groningen

Another summing example (Dafny version)

```
ghost function U(upb: int, a: array<int>): int
  requires 0 <= upb <= a.Length
  decreases a.Length - upb
  reads a
{
  if (upb >= a.Length) then 0
  else U(upb+1, a) + a[upb]
}

ghost function T(upb: int, a: array<int>): int
  requires 0 <= upb <= a.Length
  decreases a.Length - upb
  reads a
{
  if (upb >= a.Length) then 0
  else T(upb+1, a) + a[upb]*(U(upb, a) + U(upb+1, a))
}

ghost function S(upb: int, a: array<int>): int
  requires 0 <= upb <= a.Length
  decreases a.Length - upb
  reads a
{
  if (upb >= a.Length) then 0
  else S(upb+1, a) + a[upb]*T(upb, a)
}
```

university of groningen

Maximum of products

Given is the declaration of two arrays:

$$\textbf{const}\ n :\ \mathbb{N}^+;\ a, b :\ \ \textbf{array}\ [0..n)\ \textbf{of}\ \mathbb{N};$$

Determine a command that computes

$$\text{Max}\ (a[i] \cdot b[j] \mid i, j :\ 0 \leq i < j < n)$$

We introduce: $M(x) = \text{Max}\ (a[i] \cdot b[j] \mid i, j :\ 0 \leq i < j < x)$
and aim for a command S that satisfies the specification:

const $n :\ \mathbb{N};\ a, b :\ \ \textbf{array}\ [0..n)\ \textbf{of}\ \mathbb{N};$
var $m :\ \mathbb{Z};$
$\{P :\ n \geq 1\}$
S
$\{Q :\ m = M(n)\}$

university of groningen

Example: Maximum of products

We try to find a recurrence for $M(x+1)$.

$$\begin{array}{ll} & M(x+1) \\ = & \quad \{\text{definition } M\} \\ & \text{Max}\ (a[i] \cdot b[j] \mid i, j :\ 0 \leq i < j < x+1) \\ = & \quad \{\textbf{assume}\ 1 \leq x < n; \text{ split domain: } j = x\ \vee\ j < x\} \\ & \text{Max}\ (a[i] \cdot b[j] \mid i, j :\ 0 \leq i < j < x)\ \textbf{max}\ \text{Max}\ (a[i] \cdot b[x] \mid i :\ 0 \leq i < x) \\ = & \quad \{\text{definition } M\text{; calculus; note that } b[x] \geq 0\} \\ & M(x)\ \textbf{max}\ (b[x] \cdot \text{Max}\ (a[i] \mid i :\ 0 \leq i < x) \\ = & \quad \{\text{introduce } T(x) = \text{Max}\ (a[i] \mid i :\ 0 \leq i < x)\} \\ & M(x)\ \textbf{max}\ (b[x] \cdot T(x)) \end{array}$$

The base case for this recurrence is: $x \leq 1 \Rightarrow M(x) = -\infty$
Next, we have a look at $T(x+1)$:

$$\begin{array}{ll} & T(x+1) \\ = & \quad \{\text{definition } T\} \\ & \text{Max}\ (a[i] \mid i :\ 0 \leq i < x+1) \\ = & \quad \{\textbf{assume}\ 0 \leq x < n; \text{ split domain: } i = x\ \vee\ i < x\} \\ & \text{Max}\ (a[i] \mid i :\ 0 \leq i < x)\ \textbf{max}\ a[x] \\ = & \quad \{\text{definition } T\} \\ & T(x)\ \textbf{max}\ a[x] \end{array}$$

university of groningen

The base case is for this recurrence is: $x \leq 0 \Rightarrow T(x) = -\infty$

Example: Maximum of products

1 Choose an invariant J, and guard B.

$$J: \ m = M(x) \ \wedge \ t = T(x) \ \wedge \ 1 \leq x \leq n$$

Clearly, we choose $B: \ x \neq n$, such that $J \ \wedge \ \neg B \Rightarrow m = M(n)$.

2 Initialization:

$\{P: \ n \geq 1\}$
$\quad (*$ base case M; $T(1) = -\infty$ **max** $a[0] = a[0]$ $*)$
$\{-\infty = M(1) \ \wedge \ a[0] = T(1) \ \wedge \ 1 \leq 1 \leq n\}$
$m, t, x := -\infty, \ a[0], \ 1;$
$\{J: \ m = M(x) \ \wedge \ t = T(x) \ \wedge \ 1 \leq x \leq n\}$

3 Variant function: $\text{vf} = n - x \in \mathbb{Z}$. Clearly, $J \ \wedge \ B \Rightarrow \text{vf} \geq 0$

university of groningen

Example: Maximum of products

4 Body: $\{J \ \wedge \ B \ \wedge \ \text{vf} = V\} \ S \ \{J \ \wedge \ \text{vf} < V\}$

$\{m = M(x) \ \wedge \ t = T(x) \ \wedge \ 1 \leq x < n \ \wedge \ n - x = V\}$
$\quad (*$ recurrence $M(x+1) = M(x)$ **max** $(b[x] \cdot T(x))$; substitution; logic $*)$
$\{m$ **max** $(b[x] \cdot t) = M(x+1) \ \wedge \ t = T(x) \ \wedge \ 1 \leq x < n \ \wedge \ n - x = V\}$
$m := m$ **max** $(b[x] * t);$
$\{m = M(x+1) \ \wedge \ t = T(x) \ \wedge \ 1 \leq x < n \ \wedge \ n - x = V\}$
$\quad (*$ recurrence $T(x+1) = T(x)$ **max** $a[x]$; substitution $*)$
$\{m = M(x+1) \ \wedge \ t$ **max** $a[x] = T(x+1) \ \wedge \ 1 \leq x < n \ \wedge \ n - x = V\}$
$t := t$ **max** $a[x];$
$\{m = M(x+1) \ \wedge \ t = T(x+1) \ \wedge \ 1 \leq x < n \ \wedge \ n - x = V\}$
$\quad (*$ prepare $x := x + 1$ $*)$
$\{m = M(x+1) \wedge t = T(x+1) \wedge 1 \leq x+1 \leq n \wedge n - (x+1) < V\}$
$x := x + 1;$
$\{m = M(x) \ \wedge \ t = T(x) \ \wedge \ \ \wedge 1 \leq x \leq n \ \wedge \ n - x < V\}$
$\quad (*$ definitions J, and vf $*)$
$\{J \ \wedge \ \text{vf} < V\}$

university of groningen

Example: Maximum of products

5 Conclusion

const $n : \mathbb{N}$; $a, b :$ **array** $[0..n)$ **of** $\mathbb{N}$;
var $m, t : \mathbb{Z}$; $x : \mathbb{N}$;
  $\{P : n \geq 1\}$
$m := -\infty$;
$t := a[0]$;
$x := 1$;
  $\{J : m = M(x) \wedge t = T(x) \wedge 1 \leq x \leq n\}$
    $(*\ \mathrm{vf} = n - x\ *)$
while $x < n$ **do**
  $m := m$ **max** $(b[x] * t)$;
  $t := t$ **max** $a[x]$;
  $x := x + 1$;
end;
  $\{Q : m = \mathrm{Max}\,(a[i] \cdot b[j] \mid i, j : 0 \leq i < j < n)\}$

university of groningen

Maximum of products (Dafny)

```
function max(a: int, b:int): int
{
  if a >= b then a else b
}

ghost function T(limit: int, a: array<nat>) : int
 requires 0 <= limit <= a.Length
 reads a
{
    if limit <= 0 then -999
    else max(a[limit - 1], T(limit - 1, a))
}

ghost function M(limit: int, a: array<nat>, b: array<nat>) : int
 requires 0 <= limit <= a.Length == b.Length
 reads a, b
{
    if limit <= 1 then -999
    else max(M(limit - 1, a, b), b[limit - 1]*T(limit - 1, a))
}
```

university of groningen

Maximum of products (Dafny)

```
method maxProducts(a: array<nat>, b: array<nat>) returns (m: int)
 requires a.Length == b.Length >= 1
 ensures m == M(a.Length, a, b)
{
  m := -999;
  var t := a[0];
  var x:= 1;
  while x < a.Length
  invariant 1 <= x <= a.Length && t == T(x, a) && m == M(x, a, b)
  {
    m := max(m, b[x]*t);
    t := max(t, a[x]);
    x := x + 1;
  }
}
```

university of groningen

Questions?



university of groningen